

# WebSocket Send & Catch Functions - Complete Reference Table

## Core Principle

Every `send()` function on one side is caught by a corresponding event listener on the other side.

## Master Reference Table

Table 1: Native WebSocket Functions

| Action            | Client Side Function                        | Caught By Server                                         | Server Side Function                        | Caught By Client                                          |
|-------------------|---------------------------------------------|----------------------------------------------------------|---------------------------------------------|-----------------------------------------------------------|
| Send Text Message | <code>ws.send('text')</code>                | <code>ws.on('message', (data) =&gt; {})</code>           | <code>ws.send('text')</code>                | <code>ws.addEventListener('message', (e) =&gt; {})</code> |
| Send JSON         | <code>ws.send(JSON.stringify({...}))</code> | <code>ws.on('message', (data) =&gt; {})</code>           | <code>ws.send(JSON.stringify({...}))</code> | <code>ws.addEventListener('message', (e) =&gt; {})</code> |
| Send Binary       | <code>ws.send(arrayBuffer)</code>           | <code>ws.on('message', (data, isBinary) =&gt; {})</code> | <code>ws.send(buffer)</code>                | <code>ws.addEventListener('message', (e) =&gt; {})</code> |
| Connect           | <code>new WebSocket(url)</code>             | <code>wss.on('connection', (ws) =&gt; {})</code>         | N/A                                         | <code>ws.addEventListener('open', () =&gt; {})</code>     |
| Close Connection  | <code>ws.close(code, reason)</code>         | <code>ws.on('close', (code, reason) =&gt; {})</code>     | <code>ws.close(code, reason)</code>         | <code>ws.addEventListener('close', (e) =&gt; {})</code>   |
| Send Ping         | N/A (auto)                                  | <code>ws.on('ping', (data) =&gt; {})</code>              | <code>ws.ping(data)</code>                  | N/A (auto)                                                |
| Send Pong         | N/A (auto)                                  | <code>ws.on('pong', (data) =&gt; {})</code>              | <code>ws.pong(data)</code>                  | N/A (auto)                                                |
| Terminate         | N/A                                         | N/A                                                      | <code>ws.terminate()</code>                 | <code>ws.addEventListener('close', (e) =&gt; {})</code>   |
| Error Occurs      | N/A                                         | <code>ws.on('error', (error) =&gt; {})</code>            | N/A                                         | <code>ws.addEventListener('error', (e) =&gt; {})</code>   |



Table 2: Application-Level Custom Messages (JSON Type-Based)

Authentication & User Management

| Action             | Client Sends                                                                     | Server Catches                                                                                 | Server Sends                                                                      | Client Catches                                                                                     |
|--------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Set Username       | <code>ws.send(JSON.stringify({type: 'set-username', username: 'Alice'})))</code> | <code>ws.on('message', (data) =&gt; {})</code><br>then parse <code>type: 'set-username'</code> | <code>ws.send(JSON.stringify({type: 'username-set', username: 'Alice'})))</code>  | <code>ws.addEventListe</code><br><code>(e) =&gt; {})</code> then pa<br><code>'username-set'</code> |
| Username Error     | N/A                                                                              | N/A                                                                                            | <code>ws.send(JSON.stringify({type: 'error', message: 'Username taken'})))</code> | <code>ws.addEventListe</code><br><code>(e) =&gt; {})</code> then pa<br><code>'error'</code>        |
| Connection Welcome | N/A                                                                              | N/A                                                                                            | <code>ws.send(JSON.stringify({type: 'connection', message: 'Welcome'})))</code>   | <code>ws.addEventListe</code><br><code>(e) =&gt; {})</code> then pa<br><code>'connection'</code>   |

Room Management

| Action                | Client Sends                                                                 | Server Catches                                                                               | Server Sends                                                                            | Client Catches                                                                                       |
|-----------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Join Room             | <code>ws.send(JSON.stringify({type: 'join-room', room: 'general'})))</code>  | <code>ws.on('message', (data) =&gt; {})</code><br>then parse <code>type: 'join-room'</code>  | <code>ws.send(JSON.stringify({type: 'joined-room', room: 'general'})))</code>           | <code>ws.addEventListen</code><br><code>(e) =&gt; {})</code> then pars<br><code>'joined-room'</code> |
| User Joined Broadcast | N/A                                                                          | N/A                                                                                          | <code>ws.send(JSON.stringify({type: 'user-joined', username: 'Alice'})))</code> to room | <code>ws.addEventListen</code><br><code>(e) =&gt; {})</code> then pars<br><code>'user-joined'</code> |
| Leave Room            | <code>ws.send(JSON.stringify({type: 'leave-room', room: 'general'})))</code> | <code>ws.on('message', (data) =&gt; {})</code><br>then parse <code>type: 'leave-room'</code> | <code>ws.send(JSON.stringify({type: 'left-room', room: 'general'})))</code>             | <code>ws.addEventListen</code><br><code>(e) =&gt; {})</code> then pars<br><code>'left-room'</code>   |
| User Left Broadcast   | N/A                                                                          | N/A                                                                                          | <code>ws.send(JSON.stringify({type: 'user-left', username: 'Alice'})))</code> to room   | <code>ws.addEventListen</code><br><code>(e) =&gt; {})</code> then pars<br><code>'user-left'</code>   |

| Action                   | Client Sends                                                                   | Server Catches                                                                                | Server Sends                                                                           | Client Catches                                                                       |
|--------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>Create Room</b>       | <code>ws.send(JSON.stringify({type: 'create-room', room: 'new-room'})))</code> | <code>ws.on('message', (data) =&gt; {})</code><br>then parse <code>type: 'create-room'</code> | <code>ws.send(JSON.stringify({type: 'room-created', room: 'new-room'})))</code> to all | <code>ws.addEventListener(e) =&gt; {})</code> then parse <code>'room-created'</code> |
| <b>Get Room Members</b>  | N/A (sent after join)                                                          | N/A                                                                                           | <code>ws.send(JSON.stringify({type: 'room-members', members: [...]}))</code>           | <code>ws.addEventListener(e) =&gt; {})</code> then parse <code>'room-members'</code> |
| <b>Request Room List</b> | <code>ws.send(JSON.stringify({type: 'get-rooms'})))</code>                     | <code>ws.on('message', (data) =&gt; {})</code><br>then parse <code>type: 'get-rooms'</code>   | <code>ws.send(JSON.stringify({type: 'rooms-list', rooms: [...]}))</code>               | <code>ws.addEventListener(e) =&gt; {})</code> then parse <code>'rooms-list'</code>   |

## Chat Messages

| Action                              | Client Sends                                                                                   | Server Catches                                                                                    | Server Sends                                                                                              | Client Catches                                                                               |
|-------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>Send Chat Message</b>            | <code>ws.send(JSON.stringify({type: 'chat', message: 'Hello'})))</code>                        | <code>ws.on('message', (data) =&gt; {})</code><br>then parse <code>type: 'chat'</code>            | <code>ws.send(JSON.stringify({type: 'chat', username: 'Alice', message: 'Hello'})))</code> to room        | <code>ws.addEventListener(e) =&gt; {})</code> then parse <code>'chat'</code>                 |
| <b>Send Private Message</b>         | <code>ws.send(JSON.stringify({type: 'private-message', to: 'Bob', message: 'Secret'})))</code> | <code>ws.on('message', (data) =&gt; {})</code><br>then parse <code>type: 'private-message'</code> | <code>ws.send(JSON.stringify({type: 'private-message', from: 'Alice', message: 'Secret'})))</code> to Bob | <code>ws.addEventListener(e) =&gt; {})</code> then parse <code>'private-message'</code>      |
| <b>Private Message Sent Confirm</b> | N/A                                                                                            | N/A                                                                                               | <code>ws.send(JSON.stringify({type: 'private-message-sent', to: 'Bob'})))</code> to sender                | <code>ws.addEventListener(e) =&gt; {})</code> then parse <code>'private-message-sent'</code> |
| <b>Typing Indicator</b>             | <code>ws.send(JSON.stringify({type: 'typing', isTyping: true})))</code>                        | <code>ws.on('message', (data) =&gt; {})</code><br>then parse <code>type: 'typing'</code>          | <code>ws.send(JSON.stringify({type: 'typing', username: 'Alice', isTyping: true})))</code> to room        | <code>ws.addEventListener(e) =&gt; {})</code> then parse <code>'typing'</code>               |

User List & Presence

| Action                            | Client Sends                                               | Server Catches                                                                              | Server Sends                                                                                 | Client Catches                                                                                                                                                                          |
|-----------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Request<br>Online Users           | <code>ws.send(JSON.stringify({type: 'get-users'})))</code> | <code>ws.on('message', (data) =&gt; {})</code><br>then parse <code>type: 'get-users'</code> | <code>ws.send(JSON.stringify({type: 'users-list', users: [...]}))</code>                     | <code>ws.addEventLis</code><br><code>(e) =&gt; {})</code> then <code>ws.addEventLis</code><br><code>(e) =&gt; {})</code> then <code>ws.addEventLis</code><br><code>(e) =&gt; {})</code> |
| User<br>Connected<br>Broadcast    | N/A                                                        | N/A                                                                                         | <code>ws.send(JSON.stringify({type: 'user-connected', username: 'Alice'})))</code> to all    | <code>ws.addEventLis</code><br><code>(e) =&gt; {})</code> then <code>ws.addEventLis</code><br><code>(e) =&gt; {})</code> then <code>ws.addEventLis</code><br><code>(e) =&gt; {})</code> |
| User<br>Disconnected<br>Broadcast | N/A (connection close)                                     | <code>ws.on('close', () =&gt; {})</code>                                                    | <code>ws.send(JSON.stringify({type: 'user-disconnected', username: 'Alice'})))</code> to all | <code>ws.addEventLis</code><br><code>(e) =&gt; {})</code> then <code>ws.addEventLis</code><br><code>(e) =&gt; {})</code> then <code>ws.addEventLis</code><br><code>(e) =&gt; {})</code> |

Detailed Function Mapping with Code Examples

1. Connection Events

javascript

## CONNECTION ESTABLISHMENT

CLIENT SIDE

SERVER SIDE

SEND / ACTION:

```
const ws = new WebSocket('ws://localhost:3000');
```

CAUGHT BY:

```
wss.on('connection', (ws, req) => {  
  console.log('New client');  
});
```

CAUGHT BY:

```
ws.addEventListener('open', (event) => {  
  console.log('Connected');  
});
```

## 2. Text Message Events

javascript

## TEXT MESSAGE EXCHANGE

CLIENT → SERVER

CLIENT SIDE

SEND:

```
ws.send('Hello Server');
```

SERVER SIDE

CAUGHT BY:

```
ws.on('message', (data) => {  
  const text = data.toString();  
  console.log(text); // 'Hello Server'  
});
```

SERVER → CLIENT

CLIENT SIDE

CAUGHT BY:

```
ws.addEventListener('message', (event) => {  
  console.log(event.data); // 'Hello Client'  
});
```

SERVER SIDE

SEND:

```
ws.send('Hello Client');
```

### 3. JSON Message Events (Application Level)

javascript

## JSON MESSAGE - SET USERNAME

CLIENT → SERVER

CLIENT SIDE

SERVER SIDE

SEND:

```
ws.send(JSON.stringify({  
  type: 'set-username',  
  username: 'Alice'  
}));
```

CAUGHT BY:

```
ws.on('message', (data) => {  
  const msg = JSON.parse(data.toString());  
  
  if (msg.type === 'set-username') {  
    // Handle set username  
    console.log(msg.username); // 'Alice'  
  }  
});
```

SERVER → CLIENT (Response)

CLIENT SIDE

SERVER SIDE

CAUGHT BY:

```
ws.addEventListener('message', (event) => {  
  const data = JSON.parse(event.data);  
  
  if (data.type === 'username-set') {  
    console.log('Username confirmed:', data.username);  
  }  
});
```

SEND:

---

```
ws.send(JSON.stringify({  
  type: 'username-set',  
  username: 'Alice'  
}));
```

---

## 4. Room Join Events

javascript



## JSON MESSAGE - JOIN ROOM

CLIENT → SERVER

CLIENT SIDE

SERVER SIDE

SEND:

```
ws.send(JSON.stringify({  
  type: 'join-room',  
  room: 'general'  
}));
```

CAUGHT BY:

```
ws.on('message', (data) => {  
  const msg = JSON.parse(data.toString());  
  
  if (msg.type === 'join-room') {  
    // Add user to room  
    console.log('User joining:', msg.room);  
  }  
});
```

SERVER → CLIENT (Confirmation)

CLIENT SIDE

SERVER SIDE

CAUGHT BY:

```
ws.addEventListener('message', (event) => {  
  const data = JSON.parse(event.data);  
  
  if (data.type === 'joined-room') {  
    console.log('You joined:', data.room);  
  }  
});
```

SEND:

---

```
ws.send(JSON.stringify({
  type: 'joined-room',
  room: 'general'
}));
```

SERVER → ALL OTHER CLIENTS IN ROOM (Broadcast)

---

OTHER CLIENTS IN ROOM

---

SERVER SIDE

---

CAUGHT BY:

---

```
ws.addEventListener('message', (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'user-joined') {
    console.log(data.username + ' joined the room');
  }
});
```

←  
SEND (broadcast to room):

---

```
rooms.get('general').forEach(clientWs => {
  if (clientWs !== ws) {
    clientWs.send(JSON.stringify({
      type: 'user-joined',
      username: 'Alice',
      room: 'general'
    }));
  }
});
```

---

## 5. Chat Message Events

javascript

## JSON MESSAGE - CHAT MESSAGE

CLIENT → SERVER

CLIENT SIDE

SERVER SIDE

SEND:

```
ws.send(JSON.stringify({  
  type: 'chat',  
  message: 'Hello everyone!'  
}));
```

CAUGHT BY:

```
ws.on('message', (data) => {  
  const msg = JSON.parse(data.toString());  
  
  if (msg.type === 'chat') {  
    // Broadcast to room  
    console.log('Chat msg:', msg.message);  
  }  
});
```

SERVER → ALL CLIENTS IN ROOM (Including Sender)

ALL CLIENTS IN ROOM

SERVER SIDE

CAUGHT BY:

```
ws.addEventListener('message', (event) => {  
  const data = JSON.parse(event.data);  
  
  if (data.type === 'chat') {  
    displayMessage(data.username, data.message);  
  }  
});
```

SEND (broadcast to all in room):

---

```
rooms.get(userRoom).forEach(clientWs => {  
  clientWs.send(JSON.stringify({  
    type: 'chat',  
    username: 'Alice',  
    message: 'Hello everyone!',  
    timestamp: new Date()  
  }));  
});
```

---

## 6. Private Message Events

javascript

## JSON MESSAGE - PRIVATE MESSAGE

CLIENT (Alice) → SERVER

CLIENT SIDE (Alice)

SERVER SIDE

SEND:

```
ws.send(JSON.stringify({
  type: 'private-message',
  to: 'Bob',
  message: 'Secret message'
}));
```

CAUGHT BY:

```
ws.on('message', (data) => {
  const msg = JSON.parse(data.toString());

  if (msg.type === 'private-message') {
    // Find Bob's socket
    // Send only to Bob
  }
});
```

SERVER → CLIENT (Bob) - Target Recipient

CLIENT SIDE (Bob)

SERVER SIDE

CAUGHT BY:

```
ws.addEventListener('message', (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'private-message') {
    console.log('DM from:', data.from);
    console.log('Message:', data.message);
  }
});
```

←  
SEND (only to Bob):

```
bobWs.send(JSON.stringify({
  type: 'private-message',
  from: 'Alice',
  message: 'Secret message'
}));
```

SERVER → CLIENT (Alice) - Confirmation

---

CLIENT SIDE (Alice)

SERVER SIDE

CAUGHT BY:

```
ws.addEventListener('message', (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'private-message-sent') {
    console.log('Message sent to:', data.to);
  }
});
```

←  
SEND (back to Alice):

```
aliceWs.send(JSON.stringify({
  type: 'private-message-sent',
  to: 'Bob',
  message: 'Secret message'
}));
```

## 7. Close Connection Events

javascript

## CONNECTION CLOSE

CLIENT → SERVER (Client Closes)

CLIENT SIDE

SERVER SIDE

SEND/ACTION:

```
ws.close(1000, 'User left');
```

CAUGHT BY:

```
ws.on('close', (code, reason) => {  
  console.log('Client closed');  
  console.log('Code:', code); // 1000  
  console.log('Reason:', reason); // 'User left'  
});
```

CAUGHT BY (acknowledgment):

```
ws.addEventListener('close', (event) => {  
  console.log('Connection closed');  
  console.log('Code:', event.code);  
});
```

SERVER → CLIENT (Server Closes)

CLIENT SIDE

SERVER SIDE

CAUGHT BY:

```
ws.addEventListener('close', (event) => {  
  console.log('Server closed connection');  
  console.log('Code:', event.code);  
  console.log('Reason:', event.reason);  
});
```

SEND/ACTION:

```
ws.close(1000, 'Server shutting down');
```

## 8. Error Events

javascript

### ERROR HANDLING

CLIENT SIDE

SERVER SIDE

CAUGHT BY:

CAUGHT BY:

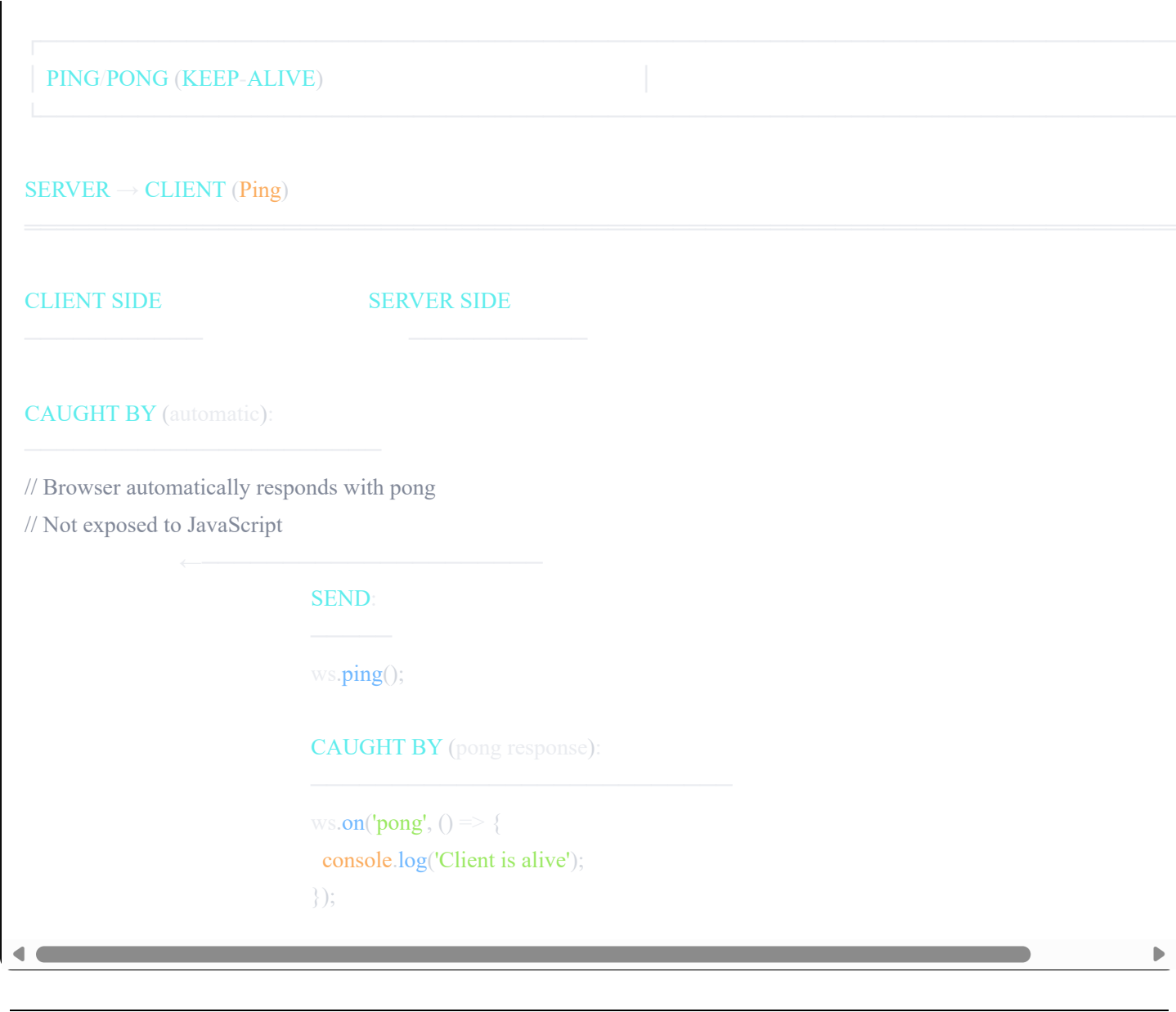
```
ws.addEventListener('error', (event) => {    ws.on('error', (error) => {  
    console.error('Client error:', event);    console.error('Server error:', error);  
});});
```

TRIGGER: Connection failures, invalid frames, network issues

## 9. Ping/Pong Events (Heartbeat)

javascript





Summary Tables

Quick Reference: Who Sends → Who Catches

Table A: Basic Operations

| Operation    | Sender | Send Function                               | Receiver | Catch Function                                        |
|--------------|--------|---------------------------------------------|----------|-------------------------------------------------------|
| Text Message | Client | <code>ws.send('text')</code>                | Server   | <code>ws.on('message', callback)</code>               |
| Text Message | Server | <code>ws.send('text')</code>                | Client   | <code>ws.addEventListener('message', callback)</code> |
| JSON Message | Client | <code>ws.send(JSON.stringify({...}))</code> | Server   | <code>ws.on('message', callback)</code>               |
| JSON Message | Server | <code>ws.send(JSON.stringify({...}))</code> | Client   | <code>ws.addEventListener('message', callback)</code> |
| Connect      | Client | <code>new WebSocket(url)</code>             | Server   | <code>wss.on('connection', callback)</code>           |
| Open Event   | Server | (handshake)                                 | Client   | <code>ws.addEventListener('open', callback)</code>    |
| Close        | Client | <code>ws.close(code, reason)</code>         | Server   | <code>ws.on('close', callback)</code>                 |
| Close        | Server | <code>ws.close(code, reason)</code>         | Client   | <code>ws.addEventListener('close', callback)</code>   |
| Ping         | Server | <code>ws.ping()</code>                      | Client   | (auto-handled)                                        |
| Pong         | Client | (auto)                                      | Server   | <code>ws.on('pong', callback)</code>                  |

**Table B: Application Messages (By Type)**

| Type Field      | Sender | Server Catches With                                        | Server Sends                                            | Client Catches With                                                     |
|-----------------|--------|------------------------------------------------------------|---------------------------------------------------------|-------------------------------------------------------------------------|
| set-username    | Client | ws.on('message')<br>→ parse → if (type === 'set-username') | N/A                                                     | N/A                                                                     |
| username-set    | Server | N/A                                                        | ws.send(JSON.stringify({type: 'username-set'})))        | ws.addEventListner('message')<br>→ parse → if (type === 'username-set') |
| join-room       | Client | ws.on('message')<br>→ parse → if (type === 'join-room')    | N/A                                                     | N/A                                                                     |
| joined-room     | Server | N/A                                                        | ws.send(JSON.stringify({type: 'joined-room'})))         | ws.addEventListner('message')<br>→ parse → if (type === 'joined-room')  |
| user-joined     | Server | N/A                                                        | ws.send(JSON.stringify({type: 'user-joined'}))) to room | ws.addEventListner('message')<br>→ parse → if (type === 'user-joined')  |
| leave-room      | Client | ws.on('message')<br>→ parse → if (type === 'leave-room')   | N/A                                                     | N/A                                                                     |
| left-room       | Server | N/A                                                        | ws.send(JSON.stringify({type: 'left-room'})))           | ws.addEventListner('message')<br>→ parse → if (type === 'left-room')    |
| user-left       | Server | N/A                                                        | ws.send(JSON.stringify({type: 'user-left'}))) to room   | ws.addEventListner('message')<br>→ parse → if (type === 'user-left')    |
| chat            | Client | ws.on('message')<br>→ parse → if (type === 'chat')         | N/A                                                     | N/A                                                                     |
| chat            | Server | N/A                                                        | ws.send(JSON.stringify({type: 'chat'}))) to room        | ws.addEventListner('message')<br>→ parse → if (type === 'chat')         |
| private-message | Client | ws.on('message')<br>→ parse → if (type                     | N/A                                                     | N/A                                                                     |

| Type Field           | Sender | Server Catches With                                       | Server Sends                                                       | Client Catches With                                                              |
|----------------------|--------|-----------------------------------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------------|
|                      |        | === 'private-message')                                    |                                                                    |                                                                                  |
| private-message      | Server | N/A                                                       | ws.send(JSON.stringify({type: 'private-message'}))) to recipient   | ws.addEventListener('message')<br>→ parse → if (type === 'private-message')      |
| private-message-sent | Server | N/A                                                       | ws.send(JSON.stringify({type: 'private-message-sent'}))) to sender | ws.addEventListener('message')<br>→ parse → if (type === 'private-message-sent') |
| typing               | Client | ws.on('message')<br>→ parse → if (type === 'typing')      | N/A                                                                | N/A                                                                              |
| typing               | Server | N/A                                                       | ws.send(JSON.stringify({type: 'typing'}))) to room                 | ws.addEventListener('message')<br>→ parse → if (type === 'typing')               |
| create-room          | Client | ws.on('message')<br>→ parse → if (type === 'create-room') | N/A                                                                | N/A                                                                              |
| room-created         | Server | N/A                                                       | ws.send(JSON.stringify({type: 'room-created'}))) to all            | ws.addEventListener('message')<br>→ parse → if (type === 'room-created')         |
| get-rooms            | Client | ws.on('message')<br>→ parse → if (type === 'get-rooms')   | N/A                                                                | N/A                                                                              |
| rooms-list           | Server | N/A                                                       | ws.send(JSON.stringify({type: 'rooms-list'})))                     | ws.addEventListener('message')<br>→ parse → if (type === 'rooms-list')           |
| get-users            | Client | ws.on('message')<br>→ parse → if (type === 'get-users')   | N/A                                                                | N/A                                                                              |
| users-list           | Server | N/A                                                       | ws.send(JSON.stringify({type: 'users-list'})))                     | ws.addEventListener('message')<br>→ parse → if (type === 'users-list')           |

| Type Field                | Sender | Server Catches With | Server Sends                                                 | Client Catches With                                                                                |
|---------------------------|--------|---------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>room-members</code> | Server | N/A                 | <code>ws.send(JSON.stringify({type: 'room-members'}))</code> | <code>ws.addEventListener('message')</code><br>→ parse → <code>if (type === 'room-members')</code> |
| <code>error</code>        | Server | N/A                 | <code>ws.send(JSON.stringify({type: 'error'}))</code>        | <code>ws.addEventListener('message')</code><br>→ parse → <code>if (type === 'error')</code>        |

The Golden Rule

||

||

|| EVERY ws.send() on ONE side ||

|| is CAUGHT BY .on('message') or .addEventListener('message') ||

|| on the OTHER side ||

||

|| The WebSocket protocol ONLY has ONE "message" event ||

|| We create MANY logical types using JSON: { type: 'xxx', ... } ||

||

Visual Summary

CLIENT FUNCTIONS

---

→ CAUGHT BY SERVER

---

→ SERVER FUNCTIONS

---

ws.send() → ws.on('message') → Parse & Route

new WebSocket() → wss.on('connection') → Setup handlers

ws.close() → ws.on('close') → Cleanup

SERVER FUNCTIONS

---

→ CAUGHT BY CLIENT

---

→ CLIENT FUNCTIONS

---

ws.send() → addEventListener('message') → Parse & Route

(handshake) → addEventListener('open') → Initialize

ws.close() → addEventListener('close') → Reconnect

ws.ping() → (auto pong) → Keep alive

# Complete Code Example with All Mappings

## Server Side - Complete Handler

javascript

```
const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 3000 });

// CATCHES: new WebSocket() from client
wss.on('connection', (ws, request) => {
  console.log('New connection');

  // CATCHES: ws.send() from client
  ws.on('message', (data) => {
    const message = JSON.parse(data.toString());

    // Route based on type
    switch (message.type) {
      case 'set-username':
        // SEND: username confirmation
        ws.send(JSON.stringify({
          type: 'username-set',
          username: message.username
        }));
        break;

      case 'join-room':
        // SEND: join confirmation
        ws.send(JSON.stringify({
          type: 'joined-room',
          room: message.room
        }));
        // SEND: broadcast to others
        broadcast({
          type: 'user-joined',
          username: getUsername(ws)
        });
        break;

      case 'chat':
        // SEND: broadcast message
        broadcastToRoom(message.room, {
          type: 'chat',
          username: getUsername(ws),
          message: message.message
        });
        break;

      case 'private-message':
        // SEND: to recipient
        sendToUser(message.to, {
```

```

    type: 'private-message',
    from: getUsername(ws),
    message: message.message
  });
  // SEND: confirmation to sender
  ws.send(JSON.stringify({
    type: 'private-message-sent',
    to: message.to
  }));
  break;
}
});

// CATCHES: ws.close() from client
ws.on('close', (code, reason) => {
  console.log('Client disconnected');
  // SEND: broadcast to others
  broadcast({
    type: 'user-left',
    username: getUsername(ws)
  });
});

// CATCHES: errors
ws.on('error', (error) => {
  console.error('Error:', error);
});

// CATCHES: pong from client
ws.on('pong', () => {
  console.log('Client is alive');
});
});

```

## Client Side - Complete Handler

```
javascript
```



```
const ws = new WebSocket('ws://localhost:3000');

// CATCHES: handshake complete from server
ws.addEventListener('open', (event) => {
  console.log('Connected');

  // SEND: set username
  ws.send(JSON.stringify({
    type: 'set-username',
    username: 'Alice'
  }));
});

// CATCHES: ws.send() from server
ws.addEventListener('message', (event) => {
  const data = JSON.parse(event.data);

  // Route based on type
  switch (data.type) {
    case 'username-set':
      console.log('Username set:', data.username);
      break;

    case 'joined-room':
      console.log('Joined room:', data.room);
      break;

    case 'user-joined':
      console.log(data.username + ' joined');
      break;

    case 'chat':
      displayMessage(data.username, data.message);
      break;

    case 'private-message':
      displayPrivateMessage(data.from, data.message);
      break;

    case 'user-left':
      console.log(data.username + ' left');
      break;

    case 'error':
      console.error('Error:', data.message);
      break;
```

```
}  
});  
  
// CATCHES: ws.close() from server OR network failure  
ws.addEventListener('close', (event) => {  
  console.log('Disconnected');  
  console.log('Code:', event.code);  
  console.log('Reason:', event.reason);  
});  
  
// CATCHES: errors  
ws.addEventListener('error', (event) => {  
  console.error('Connection error');  
});  
  
// User clicks button to join room  
function joinRoom(roomName) {  
  // SEND: join room request  
  ws.send(JSON.stringify({  
    type: 'join-room',  
    room: roomName  
  }));  
}  
  
// User types and sends message  
function sendMessage(message) {  
  // SEND: chat message  
  ws.send(JSON.stringify({  
    type: 'chat',  
    message: message  
  }));  
}  
  
// User sends private message  
function sendPrivateMessage(toUser, message) {  
  // SEND: private message  
  ws.send(JSON.stringify({  
    type: 'private-message',  
    to: toUser,  
    message: message  
  }));  
}
```

## Key Takeaways

1. Every `send()` is caught by `on('message')` or `addEventListener('message')`
2. WebSocket protocol only has ONE message event
3. We create multiple logical events using JSON with `type` field
4. Connection events are special: `connection`, `open`, `close`, `error`
5. Ping/Pong is mostly automatic for keep-alive
6. Server can broadcast to multiple clients, client can only send to server
7. All application logic happens in the `message` event handler by routing on `type`